Full Stack Development
Further Web Programming
COSC2578 / COSC2938
Semester 1, 2026

Week 02

More on React components; components interacting - part 1 & incorporating data; styles; useState

CRICOS provider number:
00122A | RTO Code: 3046

ACKNOWLEDGEMENT OF COUNTRY

RMIT University acknowledges the Wurundjeri people of the Kulin Nations as the traditional owners of the land on which the University stands.

RMIT University
respectfully recognises Elders both past and present.

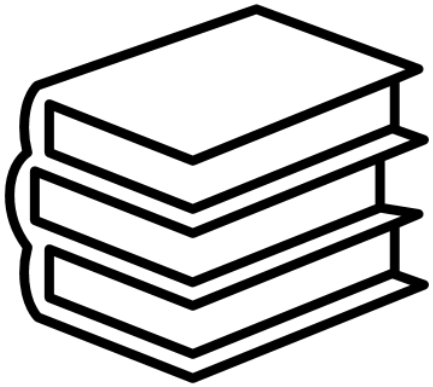
Today's agenda

House-keeping

Lectorial Segment 1

Lectorial Segment 2

Assignment 1 discussion



House keeping

Lectorial session: Monday 2:30-4:30pm

Repeat lectorial session: Friday 3:30-5:30pm

Recording available on Canvas -> Collaborate Ultra -> Recordings

- This is a fast-moving elective course, and you really need to keep up with the pace

House keeping

What needs to happen

1. You need to read Week 1 lecture notes
2. Watch the associated recording
3. Finish your Weekly Prac Exercise 01 and submit it on Github
4. Sort your Github Classroom access issue with your tutor
5. Settle in your practical class. Any timetable clash, prefers another class, please contact RMIT Student Connect (https://www.rmit.edu.au/students/support-services/student-connect/ask-about-class-timetables)

Segment 1

Hooks

More on **components**

Interaction between **components**

Hooks

State in functional components

This Photo by Unknown Author is licensed under CC BY-SA-NC

Lectorial Exercise

Research and discuss

- What is a *state* in a web application?
- Why is it needed?

State in functional components: hooks

- They let you use state and other React features without writing a class.
- They simplify a lot of code writing scenarios in React
- In-built hooks in React
  - https://reactjs.org/docs/hooks-reference.html

- Basic Hooks
  - `useState`
  - `useEffect`
  - `useContext`
- Additional Hooks
  - `useReducer`
  - `useCallback`
  - `useMemo`
  - `useRef`
  - `useImperativeHandle`
  - `useLayoutEffect`
  - `useDebugValue`

useState()

- It returns a stateful value

  const [state, setState] = useState(initialState);
- The setState function is used to update the state

  setState(newState);
- Time to look at an example
- Example05

- We will keep revisiting hooks in subsequent weeks- there is a lot to cover

useState() nuts & bolts

1. Call useState() hook to enable state in a functional component.

2. The first argument of the useState(initialValue) is the state's initial value.

3. [state, setState] = useState(initialValue) returns an array of 2 items: the state value and a state updater function.

useState() nuts & bolts

4. Invoking the state updater function setState(newState) with the new value updates the state.
    - Alternatively, you can invoke the state updater with a callback setState(prev => next), which returns the new state based on previous.

5. After the state updater is called, React makes sure to re- render the component so that the new state becomes actual.

So what is a hook?

- **What is a Hook?**
  - A *Hook* is a special function that lets you “hook into” React features. For example, *useState* is a Hook that lets you add React state to function components.
- **When would I use a Hook?**
  - If you write a function component and realize you need to add some state to it, previously you had to convert it to a class. Now you can use a Hook inside the existing function component.

Reference:

https://reactjs.org/docs/hooks-reference.html

React DOM

- Think of all of the examples covered in week 1.
- Where do you think the- *App component instantiation is occurring?*
- As discussed in week 1, it occurs in the src/pages/_app.tsx file

```
import "@/styles/globals.css";
import type { AppProps } from "next/app";

export default function App({ Component, pageProps }: AppProps) {
  return <Component {...pageProps} />;
}
```

Some details on NextJS

- In earlier versions of React, you had to use ReactDOM.render to manually mount your application to the DOM.
- Next.js abstracts this away for you. When you create pages inside the pages/ directory, Next.js automatically sets up the React component tree and mounts it to the DOM
- Removing the need for direct calls to ReactDOM.render.

```
import React from "react";
import ReactDOM from "react-dom";

function App() {
  return <h1>Hello, world!</h1>;
}

ReactDOM.render(<App />, document.getElementById("root"));
```

React component definition

- There are multiple ways of declaring a component.
- So far, we have used the function statement, though arrow functions can be used more concisely

```
// function declaration
function () { ... }

// arrow function declaration
const () => { ... }
```

```
// allowed
const item => { ... }

// allowed (recommended)
const (item) => { ... }

// not allowed
const item, index => { ... }

// allowed (recommended)
const (item, index) => { ... }
```

React component definition

- If an arrow function's only purpose is to return a value and it doesn't have any business logic in between, you can remove the block body (curly braces) of the function.

```
// with block body
const countPlusOne = (count) => {
  // perform any task in between

  return count + 1;
};

// with concise body
const countPlusOne = (count) =>
  count + 1;

// with concise body as one line
const countPlusOne = (count) => count + 1;
```

Can I use class components for this course?

- In this course we will only learn how to write new React i.e. *functional* components
- You are NOT allowed OR advised to write class components
  - You will get a zero in assessments for using class components
- Only legacy projects use class components
- So remember
  - Class components = ZERO

Lectorial Exercise

Research and discuss

- What is *legacy code*?
- How to work effectively with a legacy code?

Going back to JSX

- It is called JSX, and it is a syntax extension to JavaScript. The Typescript version is called TSX, but for simplicity we'll refer to JSX throughout this course.
- JSX produces React "elements".
- You can embed expressions in JSX const

  name = 'Jane Doe';

  const element = <h1>Hello, {name}</h1>;
- After compilation, JSX expressions become regular JavaScript function calls and evaluate to JavaScript objects.

```
function getGreeting(user) {
  if (user) {
    return <h1>Hello, {formatName(user)}!</h1>;
  }
```

Going back to JSX

- In JSX you can specify *attributes*
- JSX prevents SQL *injection attacks*
  - By default, React DOM escapes any values embedded in JSX before rendering them.
- Read more at
  - [https://react.dev/learn/writing-markup-with-jsx]

Interaction between components

- Components can refer to other components in their output.
- This lets us use the same component abstraction for any level of detail.
- Do not be afraid to split components into smaller components.
- Besides it is a bad practice to write everything in ONE BIG component
- When you separate components, they will often need to interreact with each other

Interaction between components

- Extracting components might seem like grunt work at first, but having a palette of reusable components pays off in larger apps.
- A good *rule of thumb* is that if a part of your UI is used several times, or is complex enough on its own, it is a good candidate to be extracted to a separate component.

- Example 1
  - page layout using multiple components

React Props

- Before we delve into an example, we need to understand another concept i.e.. *Props*
- “Props” is a special keyword in React, which stands for properties and is used for passing data from one component to another.
- React Props are like function arguments in JavaScript and attributes in HTML.
- But:
  - data with props are being passed in a uni-directional flow (one way from parent to child)
  - props data is read-only, which means that data coming from the parent should not be changed by child components

React Props

```
import React from "react";

interface WelcomeProps {
  name: string;
}

const Welcome: React.FC<WelcomeProps> = ({ name }) => {
  return <h1>Hello, {name}</h1>;
};

const App: React.FC = () => {
  return (
    <div>
      <Welcome name="Sara" />
      <Welcome name="Cahal" />
      <Welcome name="Edite" />
    </div>
  );
};

export default App;
```

Interaction between components

- How to use props?
- Firstly, define an attribute and its value(data)
- Then pass it within component(s) by using *props*
- Finally, render the props Data
- Time to now tie all this up with the help of a neat example
- Example 2

The rule of the props

- React has one weird strict rule:
  - *All React components must act like pure functions with respect to their props.*
  - [https://react.dev/learn/passing-props-to-a-component]
- *WHAT DOES THAT MEAN!*
- So we need to learn something that will help us make those changes
- This is where concept of “state” will come to rescue
  - State allows React components to change their output over time in response to user actions, network responses, and anything else, without violating this rule.

Handling events

- Event handling makes it possible for your users to interact with the React app
- As per React documentation- handling events with React elements is similar to handling events on DOM elements except
  - React events are named using camelCase, rather than lowercase.
  - With JSX you pass a function as the event handler, rather than a string
- If you're familiar with how events work in standard HTML and JavaScript, it should be easy for you to learn how to handle events in React.

Handling events: Functional component

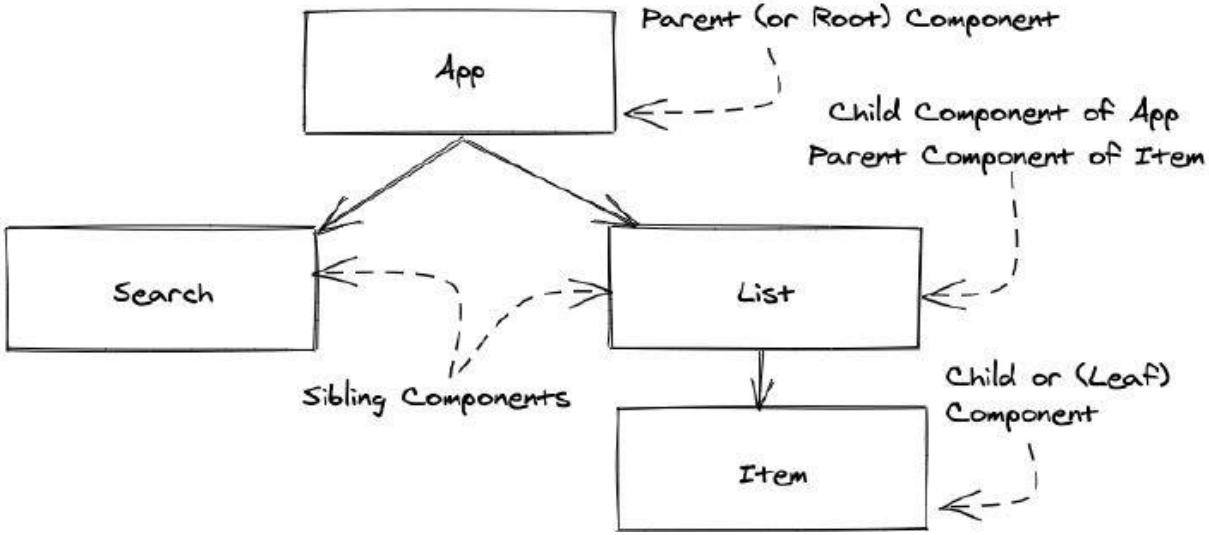
- We looked at one example in Week 1
- Here is another one
- Example 3

- We will revisit events in Week 3
- We need to look at
  - Events and state
  - Synthetic events
  - Custom events

Lectorial Exercise

Practice coding

- Complete the event handlers for **Example 3** from **Lectorial 1**.



Segment 2

Incorporating data using data structure

Styling in React

Code convention and commenting

Styling in React

- Assumption
  - You remember basic CSS syntax and concepts
  - If not, revise these

CSS in React

- Common CSS in React is similar to the standard CSS you may have already learned on your own.
- Each web application gives HTML elements a class (in React it's className) attribute that is styled via a CSS file:

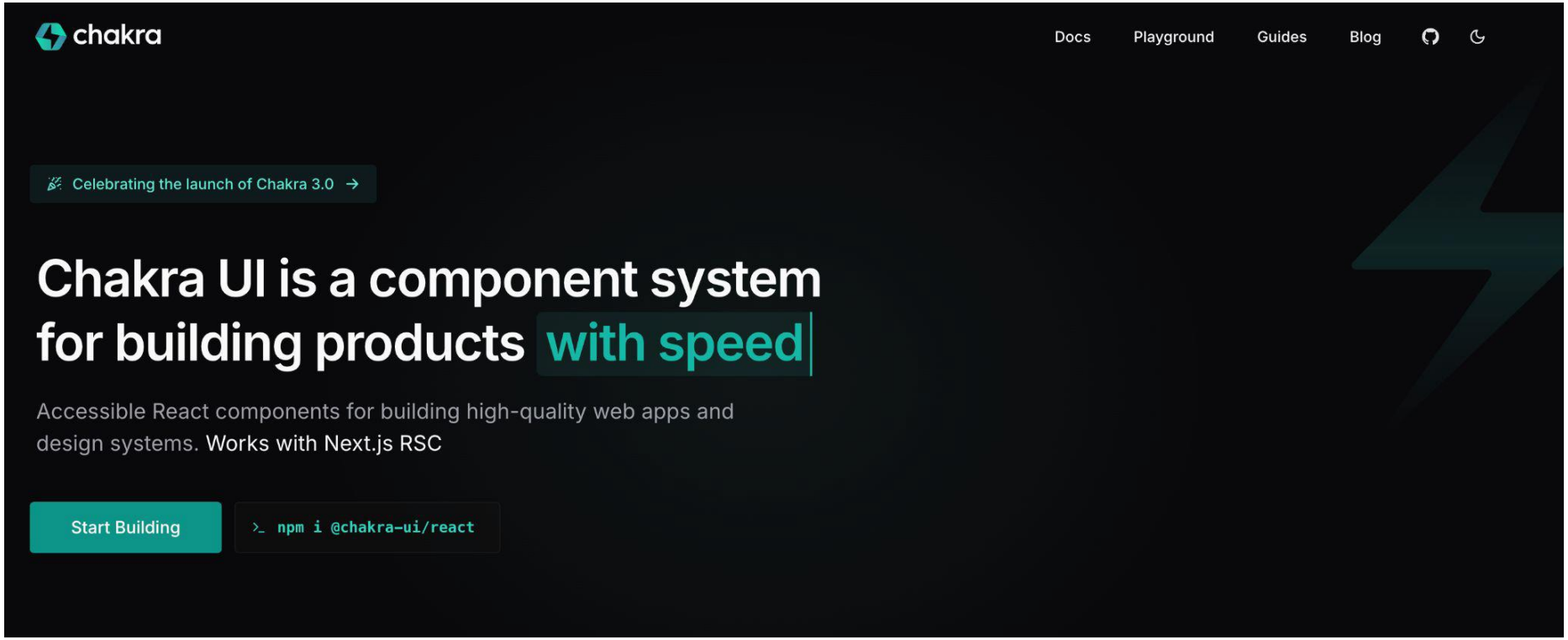
```
import "@/styles/globals.css";
```

- The CSS file is present as src/styles/globals.css
- Example 4 - Let's have a look at some basic styling

Styling in React

- You can apply styling as
  - Inline CSS properties in React code
  - Using external CSS file
  - Using CSS modules
- While you can dabble with CSS yourself, there are other ways to style a react app
  - Using a styling library, there are many but some examples
    - Chakra (chakra-ui.com)
    - Shadcn (ui.shacdn.com)
    - Tailwind (tailwindcss.com)

Chakra Example



- Example 5 - Let’s add some styled buttons

Data storage

- In any web application, you will need to store data one way or the other
- Examples- store user details, product preferences, items bought, etc.
- The data storage may be temporary or persistent
- We will learn the data storage via the databases in the latter half of this course where we will use MS SQL database *(more for assignment 2).*
- But before that, there are other ways of storing data
  - Data structures in JS
  - HTML5's localStorage object
  - *You will need these for assignment 1*

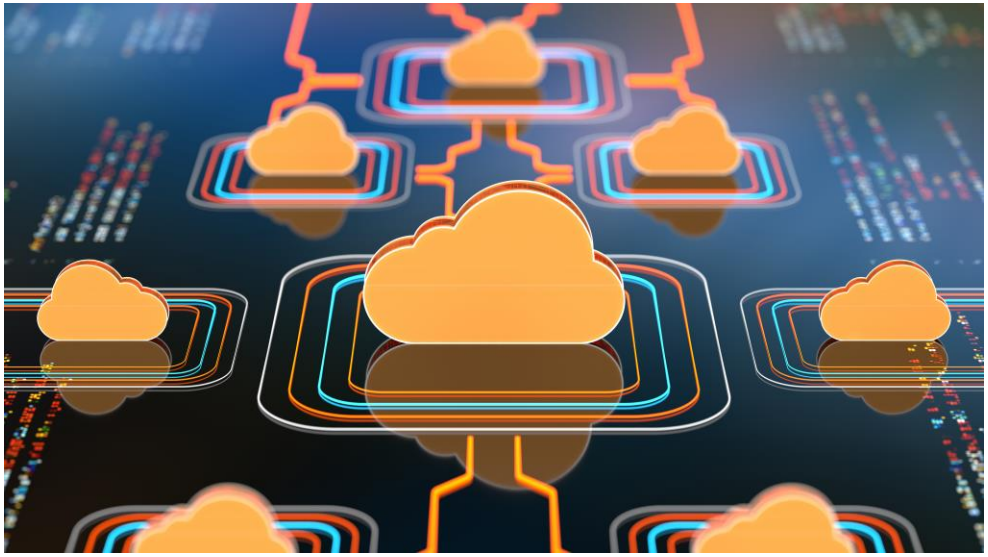
Data storage with data structures

- You can use any data structure available in JavaScript
  - Binary search tree
  - Stack
  - Queue
  - Linked List
  - Hash Table
  - Maps
  - Sets
  - JSON data
- The disadvantage being- it will be lost when you shut down the browser.

Lectorial Exercise

Research and discuss

- What is localStorage?
- What do you know about it?



Do you remember useState() *hook?*

1. Call useState() hook to enable state in a functional component.

2. The first argument of the useState(initialValue) is the state's initial value.

3. [state, setState] = useState(initialValue) returns an array of 2 items: the state value and a state updater function.

Do you remember useState() *hook?*

4. Invoking the state updater function setState(newState) with the new value updates the state.
    - Alternatively, you can invoke the state updater with a callback setState(prev => next), which returns the new state based on previous.

5. After the state updater is called, React makes sure to re-render the component so that the new state becomes actual.

Bring localStorage into the equation

- Time to look at an example – here we will use a data structure and localStorage to store the data.
- Example 6

  - A login-logout system
  - It will also be covered in the forthcoming lab

Example 6- *crib notes*

- Uses some concepts that will cover ahead
  - Forms in React
  - Conditional rendering of components
  - Using the NextJS Router - we can use this to create concepts of pages.

Example 6- *crib notes*

- The logged in username state is stored in the App parent component and is passed down to components that reference this state as props / properties.
- To modify the state functions implemented within the App component called loginUser and logoutUser are implemented. These functions are passed as props / properties to the Navbar and Login components; the Navbar component uses logoutUser and Login component uses loginUser.
- Lastly there is some conditional rendering is included in the Home and Navbar components to render different output if the user is logged in or not.

Self-exercise

- Those of you who are interested should search and learn about- how to store offline data using Dexie.js
- It uses something known as IndexedDB to store offline data
- Think about
  - Why use this option?
  - What advantages does it offer?
  - How to set it up?
  - How to use it?
- *Note: this is for enthusiastic learners- it is optional!*

Code Elegance

Writing good code

This Photo by Unknown Author is licensed under CC BY-SA

Coding convention: React

- Some of these exist- slightly different for each organisation
- These can become tedious at times to follow- *but if you end up working at a place that mandates a standard, you won't have much choice in the matter!*
- Do not think of these are *hard and fast rules*
- Adhering to a standard way of writing code leads to a professional codebase (*though it is debatable*)
- Do not pay attention to
  - Bad practices prevalent in industry
  - Cowboy coders
  - Frustrated and negative people who criticise everything
  - Arrogant, burnt out and jaded professionals

Coding convention: React

- Learn from positive, experienced people
- Often experts are invisible to amateurs-
  - someone with none or little experience (a project or 1-2 years) has a lot to learn
- Clean code writing is often hard and a regimen that arrogant developers abhor. Traits of an arrogant developer -
  - Assume that they're the smartest person in the room.
  - Refuse to explain something because the other person "wouldn't understand".
  - Talk down to others / condescend.
  - Pretend to be smarter than they really are.
  - Assume they can't learn new things from other people.

Code commenting

- One of the arguments often made by developers- *code commenting is a waste of time*
- While many developers do not use it in the right spirit- it is important to have sensible code commenting
  - When code becomes complex, sometimes the developer might not even realise how complex their code has gotten. A facile argument often overheard is-*but my code is not complex!-….* 'complex' is a relative term
  - Another really good use case for code commenting is when an anomaly occurs, say for example because of browser differences you have to do something a bit differently or have a bit of extra unusual looking code in there.

Code commenting

- Also if you have a bug and find a solution on Stack Overflow in Github issues or something, it's best to leave a link to that page in your code.
- Clarification comments are intended for anyone (including your future self) who may need to maintain, refactor, or extend your code.

- So always add comments and add them sensibly
- You are going to lose marks in assessments in absence of code commenting

Plugins for cleaner code – VS Code

- You can use plugins in VS Code
- *Prettier*-an opinionated code formatter. It obviously supports JavaScript but also many other languages like JSX, CSS, JSON or Vue.
  - It is easy to install and use
- *EsLint*- It's an open-source project initially created by Nicholas C. Zakas, which provides a pluggable linting utility for JavaScript.
- Here is a good reading:
  - [https://thomaslombart.com/setup-eslint-prettier-react]

References

- Reference: *The road to react :*The React.js with Hooks in JavaScript *(2025 edition), by Robin Weiruch; Leanpub*
- The above will be the prescribed reference textbook for the first few week(s) for this course.

Assignment 1 *first discussion*

- It is online, take-home, pair programming assignment.
- Due date: *Check Canvas*
- Worth 25%
- To be completed group of 2
- Client-side React website prototype
- Based on week(s) 1-5
- Discussion..

Next week

- More on React components
- Components interacting - part 2
- incorporating data
- State
- Forms
- props
- Styling in React